Software Architecture & Design Document (SAD)

Problem Statement No. 15 — Railway Reservation System

Project: Railway Reservation System (RRS)

Version: 1.0

Authors: Syed Obaid Faraz (PES2UG24CS546) · Simon C A (PES2UG24CS503) · Sudha (PES2UG24CS529) · Tanisha Khoria (PES2UG24CS551) — Group 5

Date: 01-10-2026

Status: Draft — Submitted for Review

Revision History

Version	Date	Author	Change Summary
1.0	01-10-2026	Group 5	Initial SAD submission for Problem Statement No. 15: Railway Reservation System

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Railway Reservation System (RRS) — the simplified core booking platform described in the project SRS. It translates the functional and non-functional requirements into a concrete component structure, interaction flows, API design, and security architecture for the development team to implement against.

1.2 Scope

Covers the RRS passenger-facing services (registration/login, train search, ticket booking and payment, cancellation, PNR status and booking history) and the admin train-management service, consistent with the SRS scope. Multi-quota fare logic, Tatkal booking, automatic waitlist/RAC promotion, and large-scale load handling remain out of scope.

1.3 Audience

Developers, QA engineers, security reviewers, and course evaluators.

1.4 Definitions

RRS (Railway Reservation System), PNR (Passenger Name Record), ADR (Architecture Decision Record), API (Application Programming Interface), JWT (JSON Web Token), TLS (Transport Layer Security), RBAC (Role-Based Access Control).

2. Document Overview

2.1 How to use this document

Provides the architectural deliverables for RRS: a component (UML) diagram, component descriptions, the chosen architecture pattern and rationale, technology stack, risks, requirement traceability, security architecture, two UML sequence diagrams for core flows, API design for two services, error-handling/logging approach, and UX notes.

2.2 Related Documents

RRS Software Requirements Specification (SRS) v1.0, RRS Software Test Plan (STP) v1.0.

3. Architecture

3.1 Goals & Constraints

Goals: A simple, correctly-functioning booking flow with no over-booking, reasonable response times for a classroom demo, and a codebase modular enough to split cleanly across feature branches.

Constraints: Small team, limited project timeline, single payment gateway (or simulated payment), and a feature set deliberately kept to the core flow defined in the SRS.

3.2 Stakeholders & Concerns

- Passengers: a simple, reliable booking experience with no double-booked seats
- Course evaluators: a clean, traceable mapping from requirements to architecture to code, and a codebase organised well enough to support clear Git/Jira workflow
- Developers (Group 5): modularity, so each module can be built and tested on its own feature branch

3.3 Component (UML) Diagram

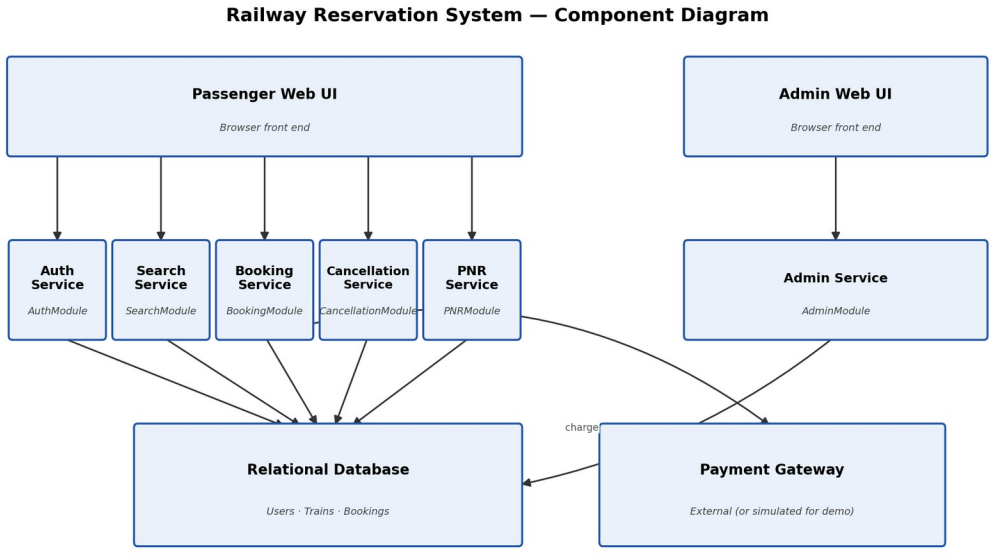


Figure 3.1 — Component Diagram (layered architecture)

3.4 Component Descriptions

- Passenger Web UI: browser front end for search, booking, cancellation, and PNR/history (RRS-F-001–041).
- Admin Web UI: browser front end for train management (RRS-F-050–052).
- Auth Service (AuthModule): registration, login, logout, password reset (RRS-F-001–004).
- Search Service (SearchModule): train search and seat/fare display (RRS-F-010–012).
- Booking Service (BookingModule): seat-availability check, payment integration, PNR generation (RRS-F-020–023).
- Cancellation Service (CancellationModule): booking cancellation and seat release (RRS-F-030–032).
- PNR Service (PNRModule): PNR status enquiry and booking history (RRS-F-040–041).
- Admin Service (AdminModule): add/update/delete trains, view bookings per train (RRS-F-050–052).
- Relational Database: stores users, trains, and bookings; single source of truth for seat counts.
- Payment Gateway: external payment provider (or a simulated payment endpoint for the demo build).

3.5 Chosen Architecture Pattern and Rationale

A layered (client → application services → database) architecture was chosen over a microservices split. The services shown in the component diagram are logical modules within a single backend application, not separately deployed services — this keeps deployment and local setup simple while still giving each module (and each Jira epic/feature branch) a clear, independent boundary. A full microservices architecture was rejected as unnecessary complexity for a project of this scope and timeline.

3.6 Technology Stack & Data Stores

Frontend: a standard web framework (e.g., React) served as static assets. Backend: a REST API (e.g., Node.js/Express or an equivalent framework) organised into the modules above. Database: a relational database (e.g., PostgreSQL/MySQL) holding Users, Trains, and Bookings tables. Communication: HTTPS between browser and backend; JWT for session authentication.

3.7 Risks & Mitigations

Risk	Mitigation
Two passengers booking the last seat at the same time (race condition)	Use a database-level constraint/transaction on seat count so only one booking can succeed (see RRS-NF-003)
Scope creep back toward the fuller original feature set	Keep new feature requests against the SRS's stated scope; log anything extra as a backlog item, not this submission

3.8 Traceability to Requirements

- RRS-F-001–004 → Auth Service
- RRS-F-010–012 → Search Service
- RRS-F-020–023 → Booking Service (+ Payment Gateway)
- RRS-F-030–032 → Cancellation Service
- RRS-F-040–041 → PNR Service
- RRS-F-050–052 → Admin Service
- RRS-NF-003 (no over-booking) → Booking Service + Database transaction

3.9 Security Architecture

Threat Modeling (STRIDE):

- Spoofing → JWT-based session authentication on every protected endpoint (RRS-SR-002)

- Tampering → server-side input validation on all booking and admin forms (RRS-SR-003)
- Repudiation → server-side logs of booking/cancellation/admin actions with timestamps
- Information Disclosure → HTTPS enforced on all connections; passwords stored hashed, never logged (RRS-SR-001, RRS-SR-005)
- Denial of Service → out of scope for this project's timeline; noted as a future enhancement
- Elevation of Privilege → Role-Based Access Control restricting admin endpoints to the Admin role (RRS-SR-004)

4. Design

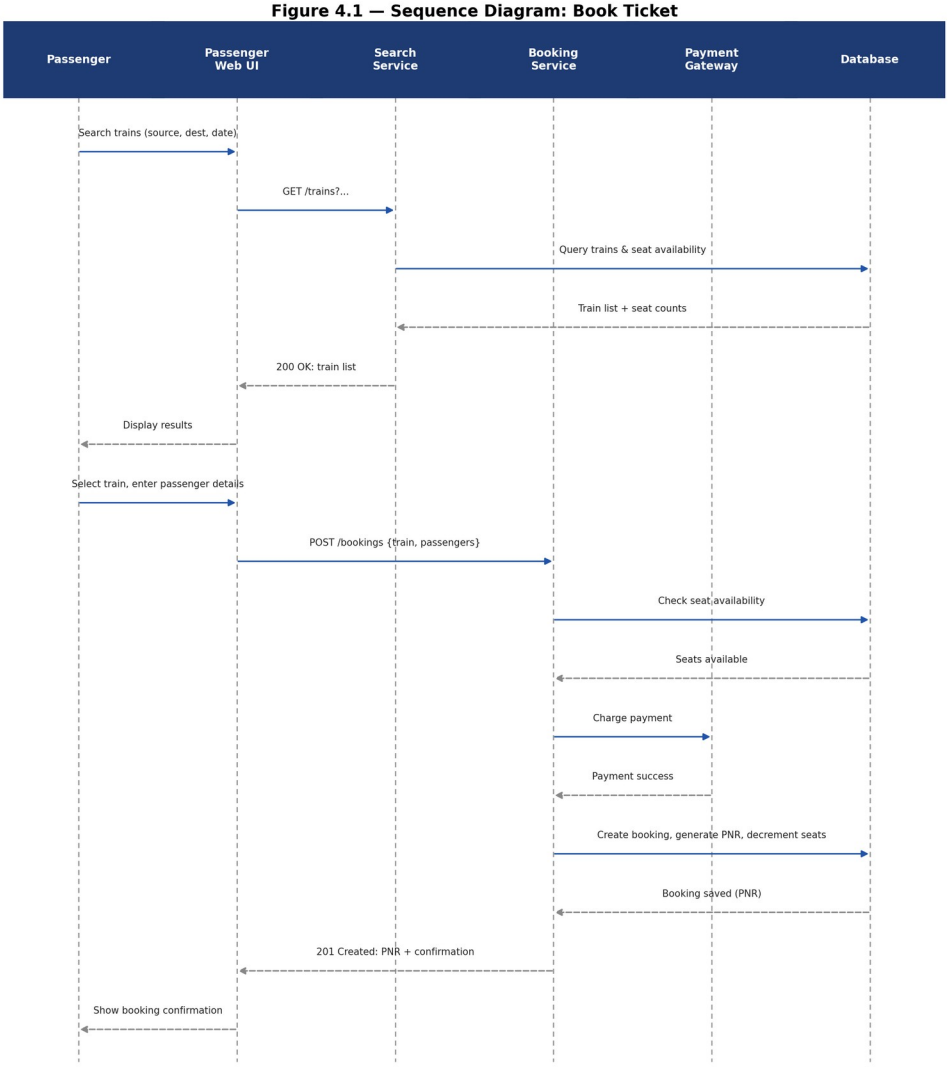
4.1 Design Overview

RRS is designed with layered modules for clear separation of concerns: presentation (web UI), application logic (the six services in Section 3.4), and data (the relational database). Each module maps to one Jira epic and one long-lived feature branch, merged into the integration branch through pull requests.

4.2 UML Sequence Diagrams

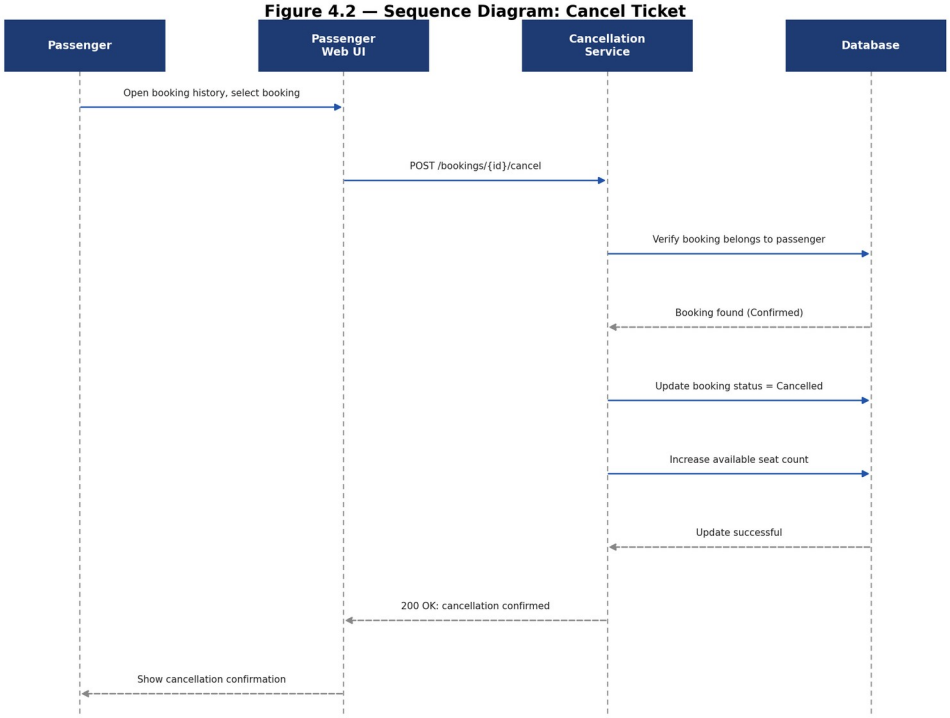
Two sequence diagrams covering the two core transactional flows of the project: booking a ticket and cancelling a ticket.

Figure 4.1 — Sequence Diagram: Book Ticket



Covers RRS-F-010–012 (search) and RRS-F-020–023 (booking & payment)

Figure 4.2 — Sequence Diagram: Cancel Ticket



Covers RRS-F-030–032 (cancellation and seat release)

4.3 API Design

Interface definitions for the Booking Service and the Cancellation Service (two components, as required).

Booking Service

Endpoint: /bookings

Method: POST

Request: { trainId, classType, passengers: [{ name, age, gender }] }

Response: { pnr, status, seatsBooked }

Errors: 400 Invalid passenger count, 401 Not authenticated, 409 Not enough seats available, 402 Payment failed

Cancellation Service

Endpoint: /bookings/{id}/cancel

Method: POST

Request: { } (booking id taken from the URL; passenger identity taken from the session token)

Response: { status: "Cancelled", refundInitiated: true }

Errors: 401 Not authenticated, 403 Booking does not belong to this user, 404 Booking not found, 409 Booking already cancelled

4.4 Error Handling, Logging & Monitoring

Standardised JSON error responses ({ error: { code, message } }) across all endpoints. No sensitive information (passwords, full card details) is ever written to logs. Basic monitoring during the project/demo period covers booking-failure rate and API error rate, checked manually rather than through a dedicated monitoring stack.

4.5 UX Design

The passenger web UI follows a simple linear wizard: search → select train/class → enter passenger details → pay → confirmation, with clear, readable error messages at each step. No specialised accessibility hardware or advanced UI framework is required beyond standard responsive, keyboard-navigable form design.

4.6 Open Issues & Next Steps

- Future enhancement: multi-quota fare logic and Tatkal booking (deferred, per SRS scope)
- Future enhancement: automatic waitlist/RAC promotion
- Future enhancement: production-grade monitoring and alerting

5. Appendices

5.1 Glossary

RRS, PNR, JWT, TLS, RBAC, ADR.

5.2 References

RRS SRS v1.0, RRS STP v1.0, IEEE 42010 (Architecture Description), OWASP Top 10, NIST SP 800-160.

5.3 Tools

Diagrams: Python/Matplotlib. API design reference: a Postman collection (to be created during implementation). Issue tracking: Jira. Version control: Git/GitHub.